Learning to Offload: Co-Adaptation Between Transformers and Cognitive Coprocessors

Working Draft

August 2026

Abstract

Study whether models trained with permanent coprocessors learn a different division of labor: better formulation and result use with less redundant internal emulation.

1 Motivation

Most LLM systems expose external computation through explicit tool invocation: a model or harness selects a function, serializes arguments, executes it, and serializes the result back into context. This series studies a different architecture in which the Transformer is one subsystem of a heterogeneous cognitive machine. Specialized engines can recognize computational opportunities, maintain structured state, derive consequences, and make useful results available to subsequent neural computation. Conventional tool calling remains a universal fallback.

2 Core Hypothesis

Tightly coupled specialized computation can improve reliability, efficiency, and scaling without abandoning Transformers as general learned semantic machines. The decisive evidence is not isolated benchmark accuracy but improved scaling with problem size under matched model and external-capability budgets.

3 Paper-Specific Objectives

- Train matched with/without-coprocessor regimes.
- Measure offloading, redundant computation, result use, and robustness.
- Probe learned representations cautiously.
- Sweep model sizes.
- Ablate delayed/noisy/incorrect coprocessor outputs and coprocessor removal.

4 Common Architecture

The program studies progressively tighter versions of

$$\text{Transformer stream} \to \text{recognizer/semantic event} \to \text{typed micro-IR} \to \text{coprocessor} \to \text{derived micro-state} \to \text{Transformer}.$$

A semantic interrupt is an automatically detected opportunity for specialized computation. A coprocessor is a persistent or reflex computational subsystem rather than merely an API selected through natural-language description.

5 Evaluation

Use matched checkpoints and tasks. Report final-answer accuracy and component-level correctness, generated tokens, model calls, latency, intervention frequency, false interventions, engine work, context/state size, and scaling curves. Include no-coprocessor and conventional explicit-tool baselines whenever applicable. Oracle trigger/selection controls should separate interface failures from engine capability.

6 Falsification

The proposed mechanism is not supported if gains disappear under matched compute and capability controls, if intervention errors offset reliability improvements, or if conventional explicit tool calling provides equivalent quality and cost without tighter coupling.

7 Scope Guardrails

- No mechanistic capacity-reallocation claim without probes.
- Keep engine correctness separate from interface learning.
- Always evaluate no-coprocessor fallback.

8 Series Roadmap

- Paper 0: Heterogeneous Cognitive Transformers: From Tool Use to Cognitive Coprocessors.
- Paper 1: Reflex Computation: Automatic Calculator Assistance During Autoregressive Decoding.
- Paper 2: Heterogeneous Reflex Reasoning: Multiple Symbolic Coprocessors with Typed Micro-State.
- Paper 3: Learned Semantic Interrupts for Cognitive Coprocessors.
- Paper 4: Transactional Cognitive State: Hypotheses, Provenance, Retraction, and Backtracking.
- Paper 5: Native Coprocessor Context: From Text Reinjection to Structured and KV-Level Interfaces.

- Paper 6: Learning to Offload: Co-Adaptation Between Transformers and Cognitive Coprocessors.
- Paper 7: A Heterogeneous Cognitive Runtime for Language Models.

9 Related Work

Toolformer learns to insert API calls into language-model generation. Neuro-symbolic systems such as AlphaGeometry demonstrate complementary neural guidance and symbolic deduction. Dynamic solver-composition work studies selection among formal reasoning paradigms, while recent verifiable neuro-symbolic programming makes reasoning explicit as compositions of primitives. This series focuses specifically on automatic, persistent, decoding-coupled cognitive coprocessors and progressively learned interfaces between neural and structured computation.

References

[1] Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS, 2023.

[2] Trinh et al. Solving Olympiad Geometry without Human Demonstrations. Nature, 2024.

[3] Xu et al. Adaptive LLM-Symbolic Reasoning via Dynamic Logical Solver Composition. EACL, 2026.

[4] Bhat et al. Forethought: Verifiable Reasoning from Neurosymbolic Primitive Programming. arXiv:2607.04096, 2026.